

SBI 811

Computer Programming in Bioinformatics

Course Outline | Master of Science in Bioinformatics

Prepared by Clabe Wekesa

Course duration	10 teaching weeks	Contact time	3 hours per week (30 hours)
Entry level	Complete beginners	Core technologies	Linux, Bash, Git, GitHub and Python
Practical focus	Bacterial RNA-seq using BOTAS	Course approach	Hands-on, reproducible and AI-integrated

1. Course description

This course introduces fundamental computer-programming concepts and their application in bioinformatics. Students learn to work in a Linux environment, use Bash to manage and inspect biological data, track work with Git and GitHub, and write Python programs for sequence and text-based biological data. The course culminates in a small, reproducible bacterial RNA-seq analysis using BOTAS.

Generative artificial intelligence is integrated as a programming assistant throughout the course. Students learn to request, inspect, modify, test and document AI-generated code while remaining responsible for computational correctness, biological validity and reproducibility.

AI may generate code, but the student must understand, test, verify and defend it.

2. Course purpose

The course is designed for MSc students entering bioinformatics without prior programming experience. It develops enough independent competence for students to use AI and specialist bioinformatics tools intelligently, rather than treating generated code or software output as automatically correct.

3. Learning outcomes

1. Navigate directories and manage files using the Linux command line.
2. Use Bash commands, pipes, redirection and simple scripts to process biological data.
3. Use Git and GitHub to obtain materials, track changes and document computational work.
4. Explain variables, data types, conditions, loops, collections and functions.
5. Write simple Python programs for DNA, RNA, protein and tabular-text data.
6. Read and process basic FASTA, FASTQ, GFF and count-table content.
7. Diagnose syntax, runtime and logical errors and construct basic tests.

8. Critically evaluate, correct and document AI-generated Bash and Python code.
9. Run a basic bacterial RNA-seq workflow with BOTAS and inspect its outputs.
10. Organize and explain a reproducible bioinformatics project in GitHub.

4. Teaching and learning approach

Each three-hour session combines a concise conceptual introduction, instructor demonstration, guided practical work, an AI-assisted challenge and a short review. Students are expected to practise between sessions and maintain their work in the course GitHub repository.

Component	Approximate time
Concepts and explanation	40 minutes
Instructor demonstration	30 minutes
Guided practical	70 minutes
AI-assisted challenge	25 minutes
Review and formative assessment	15 minutes

5. Weekly course schedule

Week	Topic	Core content	Practical outcome
1	Linux and WSL foundations	Terminal, shell, paths, files, directories, help and basic biological file types.	Set up and navigate a structured bioinformatics project.
2	Git, GitHub and project organization	Repository, clone, status, add, commit, pull, push, README and .gitignore.	Obtain materials and submit a first tracked exercise.
3	Bash for bioinformatics	Input/output, pipes, redirection, grep, cut, sort, uniq and introductory awk.	Inspect and summarize FASTA, GFF and count files.
4	Python foundations	Scripts, variables, strings, numbers, operators, print, comments and error messages.	Calculate sequence length and GC percentage.
5	Conditions, loops and collections	if/elif/else, for loops, lists, dictionaries, indexing and membership.	Validate sequences and calculate nucleotide/codon frequencies.
6	Functions, debugging and testing	Functions, parameters, return values, tracebacks, assertions and edge cases.	Build and test reusable sequence-analysis functions.
7	Reading bioinformatics files with Python	File handling, line parsing, FASTA records, GFF fields and TSV output.	Create sequence, annotation and count summaries without pandas.
8	Bacterial RNA-seq and BOTAS	Reads, reference, annotation, alignment, quantification, count tables and reproducibility.	Identify BOTAS inputs, stages, outputs and checkpoints.
9	BOTAS practical	Indexing, alignment, quantification, logs, mapping summaries and gene counts.	Run a small bacterial RNA-seq analysis and summarize results.
10	Reproducible project and defence	Project structure, documentation, verification, interpretation and AI disclosure.	Complete and defend a GitHub-based RNA-seq project.

6. Assessment strategy

Assessment combines independent foundational competence with realistic AI-assisted programming and reproducible bioinformatics work.

Assessment	Weight	Purpose
Weekly practical exercises	20%	Regular application of Linux, Git, Bash and Python skills.
Independent programming practical	20%	Demonstration of essential individual competence without generative AI.
AI-generated code audit	15%	Identification, explanation, correction and testing of computational and biological errors.
BOTAS RNA-seq project	35%	Reproducible execution, Bash/Python summarization, verification, interpretation and documentation.
Oral code defence	10%	Explanation and modification of submitted code and analytical decisions.

Assessment modes

Mode	AI rule	Educational purpose
A - Independent	No generative AI	Demonstrate essential personal competence.
B - AI-supported	AI permitted for explanation and debugging	Develop critical supported learning.
C - Open AI	Full use with disclosure and verification	Simulate responsible professional practice.

7. BOTAS capstone practical

Students will use a small, instructor-prepared bacterial dataset so that the analysis can complete during a practical session. The exercise connects the course's command-line, programming, testing and reproducibility skills.

The expected workflow is: FASTQ reads and matching reference files -> BOTAS indexing -> alignment -> gene quantification -> count table -> Bash/Python summary -> documented GitHub project.

Assessment will emphasize correct inputs, successful and reproducible execution, inspection of logs and outputs, basic biological interpretation, and clear documentation. Students are not assessed merely on whether a command runs.

8. Responsible AI-use policy

Generative AI may be used only at the level specified for each activity. Students remain responsible for every submitted command, program, output and scientific interpretation. Material AI assistance must be disclosed. Students must be able to explain, test and modify their submitted work during a practical or oral assessment.

Confidential, identifiable, clinical or unpublished research data must not be uploaded to unauthorized AI services. AI output must not be treated as an authoritative source and must be checked against documentation, known test cases and biological expectations.

9. Learning resources and software

- Windows Subsystem for Linux 2 with Ubuntu 24.04 LTS
- Anaconda Distribution and Python
- Git and GitHub course repository
- Bash and standard Linux command-line utilities
- BOTAS bacterial RNA-seq toolkit